



FeedGuard: Online Critic-Guided Reinforcement Learning with Privacy-Preserving Feedback for Recommendation

Mengying Zhu
School of Software
Technology,
Zhejiang University
Ningbo, China
mengyingzhu@zju.edu.cn

Feiyue Chen
School of Software
Technology,
Zhejiang University
Ningbo, China
fychen02@zju.edu.cn

Lifan Jiang
School of Software
Technology,
Zhejiang University
Ningbo, China
jianglf@zju.edu.cn

Mengyuan Yang*
Zhejiang University
Hangzhou, China
yangmy412@zju.edu.cn

Yangyang Wu
School of Software
Technology,
Zhejiang University
Ningbo, China
zjuwuyy@zju.edu.cn

Guanjie Cheng
School of Software
Technology,
Zhejiang University
Ningbo, China
chengguanjie@zju.edu.cn

Xiaolin Zheng
Zhejiang University
Hangzhou, China
xlzheng@zju.edu.cn

Abstract

Reinforcement learning-based recommendation systems (RLRS) are increasingly favored for their ability to leverage online interactive feedback, enabling adaptive and personalized decision-making. In this setting, user feedback serves as both a behavioral signal and an optimization target, making it essential for policy learning. However, collecting such feedback, e.g., clicks, ratings, and engagement traces, raises serious privacy concerns, posing critical challenges for value estimation, online adaptation, and privacy protection. In this paper, we propose FeedGuard, a critic-guided reinforcement learning framework with privacy-preserving feedback. FeedGuard enhances trajectory modeling via critic guidance, enables joint online fine-tuning with effective exploration-exploitation trade-offs, and enforces end-to-end privacy protection across the feedback lifecycle via split federated learning and differential privacy. We further provide a formal analysis of its differential privacy guarantees. Extensive experiments on four public recommendation datasets and the VirtualTB platform show that FeedGuard performs well in both offline and online settings, while maintaining rigorous privacy guarantees with minimal degradation.

CCS Concepts

• **Information systems** → **Recommender systems; Users and interactive retrieval**; • **Security and privacy** → *Domain-specific security and privacy architectures*.

Keywords

Recommendation, Reinforcement Learning-based Recommendation System (RLRS), Decision Transformer, Federated Learning

*Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates.

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792155>

ACM Reference Format:

Mengying Zhu, Feiyue Chen, Lifan Jiang, Mengyuan Yang, Yangyang Wu, Guanjie Cheng, and Xiaolin Zheng. 2026. FeedGuard: Online Critic-Guided Reinforcement Learning with Privacy-Preserving Feedback for Recommendation. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, Dubai, UAE, 12 pages. <https://doi.org/10.1145/3774904.3792155>

Resource Availability:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.17288305>.

1 Introduction

Recommendation systems are a representative application of graph modeling, as they naturally form large-scale bipartite or heterogeneous interaction graphs between users and items, where effective learning strongly depends on capturing interaction structures. Reinforcement learning-based recommendation systems (RLRS) [6, 30, 37] have been widely adopted in domains such as e-commerce [4] and online advertising [3]. Unlike traditional sequential recommendation methods [16, 27] that rely on supervised learning and predefined labels for top-k evaluation, RLRS adopt an interactive paradigm where agents continuously engage with users and improve policies based on accumulated rewards over user trajectories. While this setting complicates evaluation due to the lack of fixed correct answers [7], it enables the use of long-term feedback signals, such as retention rate [37] and user satisfaction [32], which better captures users' evolving needs. In this framework, user feedback (e.g., clicks, skips, dwell time) serves a dual role: both as a behavioral signal and as an optimization target, making it essential for policy learning. However, collecting and exploiting such feedback introduces critical challenges in value estimation, online adaptation, and privacy protection, motivating the need for a new solution.

Building on this observation, we identify three specific challenges that must be addressed for RLRS to be effective in practice.

First, *existing Decision Transformer-based recommendation systems suffer from the inconsistency between sampled target returns and expected returns of actions (Challenge I)*. Decision Transformer

(DT) [5] formulates reinforcement-learning-based recommendation as return-conditioned sequence modeling, predicting future actions based on desired returns. However, recent DT-based recommendation studies [30, 33, 37] reveal a limitation: trajectories with higher observed returns do not necessarily indicate better actions, as they may result from stochastic transitions or random user behaviors. By treating the return-to-go as a deterministic quantity tied to a single trajectory, DT neglects the broader distribution of possible future outcomes [2], which leads to inconsistency between the target and expected returns. This inconsistency leads to overfitting on “lucky” trajectories and produces unstable value estimation, particularly in stochastic or suboptimal feedback environments.

Second, *online fine-tuning of RLRS remains brittle without principled mechanisms to balance exploration and exploitation (Challenge II)*. In real-world recommendation scenarios, policies must be continuously adapted to evolving user behaviors. Existing online RLRS methods [11, 22] often suffer from data inefficiency, particularly in cold-start or sparse-feedback regimes, where they need to balance short-term exploration with long-term exploitation. This challenge is further amplified in graph-structured recommendation environments, where user-item interactions form large-scale bipartite graphs and the exploration space expands combinatorially with the number of users and items. Without explicit mechanisms to regulate this trade-off, online adaptation may converge prematurely or deliver suboptimal recommendations.

Third, *user feedback privacy is at risk across the entire feedback lifecycle, from offline trajectories to online interaction traces (Challenge III)*. User feedback inherently contains sensitive information: offline trajectories may expose personal preferences, reward estimation can leak user intent, and online traces risk revealing real-time behavioral dynamics [1, 9]. These risks are particularly critical in large-scale recommendation scenarios where user-item interactions are modeled as graph-structured data. In such bipartite graphs, privacy must be preserved on both sides because user data should not leak to item holders, and item-related contextual features should remain isolated from individual user identities. However, existing recommendation systems largely rely on centralized processing or isolated protections such as secure aggregation [20, 36], which fail to provide end-to-end privacy guarantees across all training stages and do not fully comply with major data protection regulations such as the GDPR [29] and the CCPA [12].

To address these challenges, we propose FeedGuard, a critic-guided reinforcement learning framework with privacy-preserving feedback. As illustrated in Figure 1, FeedGuard reframes the recommendation process under a federated split architecture that safeguards user feedback across the full lifecycle, covering offline pretraining, reward modeling, and online fine-tuning, while preserving model scalability and adaptability. (1) In the *offline pre-training* stage, FeedGuard performs policy initialization over historical trajectories using a Decision Transformer while separating computation between clients and the server. Clients retain embedding and head layers for on-device feature processing, whereas the server hosts only the shared transformer backbone. This split design prevents raw behavioral data and latent representations from leaving the client, reducing risks of PII exposure and preference leakage. (2) During *asymmetric reward training*, FeedGuard

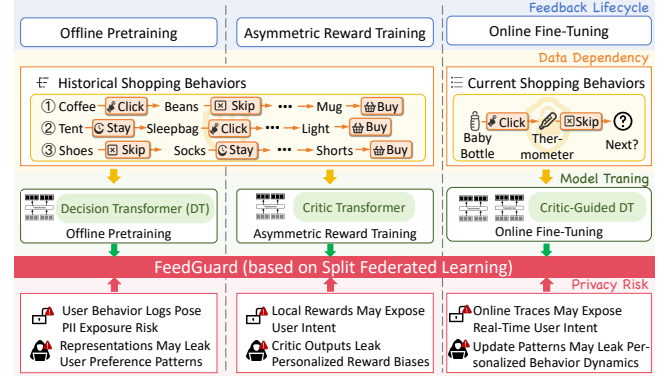


Figure 1: The feedback lifecycle in FeedGuard includes three stages, each introducing distinct privacy risks related to data dependency and model updates.

constructs a local critic transformer on each client to estimate reward signals under strict data isolation. The critic provides value guidance to the transformer backbone, directly addressing **Challenge I** while preserving privacy through differentially private transmission of intermediate representations and gradients. (3) In the *online fine-tuning* stage, FeedGuard enables joint adaptation of actor and critic models through asynchronous federated updates. Entropy regularization is introduced to maintain a principled exploration-exploitation balance, effectively mitigating **Challenge II**. Together, these three stages form an end-to-end privacy-preserving reinforcement learning pipeline (for addressing **Challenge III**). By enforcing split federated execution throughout the lifecycle, FeedGuard ensures that sensitive feedback never leaves the client side, providing rigorous differential privacy guarantees while sustaining strong recommendation performance in both offline and online settings. Our main contributions are as follows:

(1) *Paradigm*: We propose FeedGuard, a unified paradigm for critic-guided reinforcement learning with privacy-preserving feedback. To the best of our knowledge, this is the first work to localize both policy optimization and reward modeling on the client for end-to-end privacy protection.

(2) *Model*: FeedGuard introduces a critic-guided Decision Transformer that integrates explicit value estimation into trajectory modeling. The critic provides long-term return guidance, and a joint actor-critic fine-tuning scheme with entropy regularization enables stable online adaptation under dynamic user behaviors.

(3) *Privacy*: We design a split federated learning architecture that localizes sensitive computations, including embedding, critic construction, and model updates, on the client side. Only differentially private intermediate representations are transmitted to the server, ensuring end-to-end privacy protection throughout the entire feedback lifecycle. We further provide a principled approach to privacy budget allocation across training stages and communication rounds.

(4) *Experiment*: Extensive experiments show that FeedGuard achieves a 6.36% average improvement over all baselines in offline settings, and a 9.89% increase in online recommendation performance on VirtualTB. Under privacy protection, FeedGuard incurs only a 3.43% CTR degradation, which remains within an acceptable range for practical deployment in privacy-sensitive RLRS.

2 Related Work

Our research spans three interrelated directions: reinforcement learning-based recommender systems, federated reinforcement learning, and privacy-preserving recommendation. FeedGuard builds on all three by integrating critic-guided decision transformers with federated and differentially private training, thereby simultaneously improving recommendation quality and safeguarding user feedback privacy.

2.1 Reinforcement Learning-based Recommender Systems (RLRS)

RLRS formulate recommendations as a Markov Decision Process, aiming to maximize long-term user satisfaction. Existing approaches can be broadly categorized into online and offline methods.

Online RLRS adapt to evolving user preferences through online interactions, enabling dynamic policy refinement. For example, RL-radio [22] integrates explicit user preferences with implicit feedback to continuously optimize recommendation policies, thereby overcoming the inherent limitations of traditional supervised systems in fully exploiting real-time signals.

Offline RLRS, in contrast, rely on historical interactions to enhance recommendation efficiency while simultaneously mitigating the risks and costs of online exploration. A representative example is DORL [11], which balances user interest modeling and recommendation diversity by relaxing pessimism in high-entropy states. However, existing value-based offline methods still suffer from unreliable and lagged value estimation. To mitigate this issue, a series of Decision Transformer (DT)-based offline methods have been proposed. DT4Rec [37] and DT4IER [21] enhance long-term retention via return-conditioned modeling and retention-aware objectives. CDT4Rec [30] reduces bias by causal reward estimation, while EDT4Rec [6] improves generalization with reward relabeling and trajectory stitching. These methods aim to more effectively align recommendation policies with users' sustained preferences.

Nevertheless, existing RLRS methods generally overlook the critical challenge of privacy protection throughout the recommendation process. This issue remains largely unaddressed and must be tackled to ensure the feasibility, safety, and compliance of RLRS in real-world applications.

2.2 Federated Reinforcement Learning

Federated learning (FL) has been increasingly combined with reinforcement learning to preserve data privacy during distributed training in both online and offline settings.

In the online setting, FL protects user interaction data while enabling collaborative policy optimization. Qi et al. [23] provide a comprehensive survey of federated reinforcement learning, summarizing how classical RL can be integrated with FL frameworks such as FedAvg. Representative methods include FedFormer [13], which employs transformer attention to capture inter-client dependencies; FRS [15], which leverages reward shaping for information sharing; and FedKL [34], which addresses convergence issues under heterogeneous data via KL-divergence regularization.

In the offline setting, FL safeguards sensitive historical trajectories across decentralized clients. FDTR [40] formulates multi-site MDPs under linear assumptions, while FEDORA [24] removes this

constraint and introduces periodic aggregation, thereby supporting long-horizon collaborative optimization in practical environments.

Despite these advances, existing federated RL methods remain largely algorithmic prototypes that emphasize convergence or stability, but lack adaptation to the complexity of real-world recommender systems. In particular, they seldom address the intertwined challenges of feedback sparsity, client-side resource constraints, and privacy risks, making them far from ready-to-deploy solutions in practical RLRS applications.

2.3 Privacy-preserving Recommender Systems

Privacy-preserving recommender systems (PPRS) have been explored in several classical recommendation scenarios, where user interaction data may leak sensitive information.

In single-domain recommendation, privacy mechanisms have been incorporated into collaborative filtering and matrix factorization to protect user ratings and profiles while maintaining accuracy [17, 31, 35, 39]. In cross-domain recommendation, privacy has been enforced during knowledge transfer across domains or platforms, preventing leakage of user preferences while enabling collaborative utility [18]. In sequential recommendation, recent works apply privacy-preserving strategies to user-item interaction sequences, as in FMSS [20] and FELLAS [36], which enhance protection for user-merchant and user-platform interactions.

Despite these efforts, existing PPRS mainly focus on short-term prediction accuracy. To the best of our knowledge, there is still no work on end-to-end privacy protection for online reinforcement learning-based recommendation, where sensitive feedback signals play a dual role as both model inputs and optimization targets.

3 Method

We begin with the problem statement and the Decision Transformer-based Recommendation System (DTRS) that underpins our study. Building on this foundation, we present FeedGuard, detailing its three-stage training lifecycle—*offline pretraining*, *asymmetric reward training*, and *online fine-tuning*—together with the split federated execution that decouples client and server computation. Within this lifecycle, we integrate privacy-preserving mechanisms and specify how privacy budgets are allocated across stages as part of the training pipeline.

3.1 Problem Formulation

A recommender system continuously interacts with a set of users $\mathcal{U}$ and provides items from a set of candidates $\mathcal{V}$. During each interaction, the system observes the current context of a user $u \in \mathcal{U}$ and recommends item $v \in \mathcal{V}$. The user then reacts to the recommendation, e.g., by clicking, skipping, or purchasing, producing feedback that reflects the quality of the recommendation. Such interactions evolve sequentially over time and can be naturally formulated as a Markov Decision Process (MDP), to enable reinforcement learning-based optimization. Specifically, each component in the MDP tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$ corresponds to a concrete concept in the recommendation scenario:

- **State $\mathcal{S}$:** A state $s_t \in \mathcal{S}$ represents the observable context of user u at time step t , which may include the user's recent interaction

history $\mathcal{V}_u = \{v_0^u, v_1^u, \dots, v_t^u\}$, inferred preferences, and optional side information such as demographics or session-level features.

- **Action** $\mathcal{A}$: An action $a_t \in \mathcal{A}(s_t)$ corresponds to recommending an item from the candidate set $\mathcal{V}$ given the current state s_t .
- **Transition** $\mathcal{P}$: The transition function $p(s_{t+1} | s_t, a_t)$ models the user's behavioral shift after receiving the recommendation.
- **Reward** $\mathcal{R}(s_t, a_t)$: The reward quantifies user feedback (e.g., clicks, dwell time, ratings, purchases) received after recommending item a_t in state s_t , serving as an immediate signal of recommendation effectiveness.
- **Discount Factor** $\gamma \in [0, 1]$: Controls the relative importance of short-term engagement versus long-term user satisfaction.

Under this formulation, the recommender (agent) learns a policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$ that maximizes the expected cumulative return:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right], \quad (1)$$

where $\tau = (s_0, a_0, r_0, s_1, \dots)$ denotes a user-item interaction trajectory collected under policy π .

To effectively optimize this objective in high-dimensional, sparse-feedback environments, we follow the Decision Transformer (DT) paradigm, which reformulates policy learning as sequence modeling conditioned on target returns.

Each user trajectory τ is represented as a sequence of triplets:

$$\tau = (g_1, s_1, a_1, g_2, s_2, a_2, \dots, g_T, s_T, a_T), \quad (2)$$

where g_t denotes the return-to-go (RTG), i.e., the remaining cumulative reward from timestep t onward.

At each step, the DT attends to the most recent K context tokens $s_{t-K:t}, g_{t-K:t}, a_{t-K:t}$ to predict the next action (item) distribution:

$$\pi_{DT}(a_t | s_{t-K:t}, g_{t-K:t}, a_{t-K:t}), \quad (3)$$

thereby enabling the model to generate recommendations that maximize long-term user satisfaction while continuously adapting to evolving behavioral feedback.

3.2 Architecture of FeedGuard

To ensure end-to-end privacy protection throughout the feedback lifecycle, FeedGuard adopts a split federated learning architecture that separates model computation across three collaborative modules: the *clients*, the *compute server*, and the *federated server*. This modular design prevents raw user feedback and intermediate representations from leaving local devices, enabling scalable training and online adaptation under strict privacy constraints. To facilitate understanding, we first introduce the key model components of FeedGuard and their interactions, followed by how these components are distributed across the collaborative modules.

Model Components. As illustrated in Figure 2, FeedGuard consists of three core components: the *Decision Transformer*, the *Critic Transformer*, and the *Replay Buffer*.

The *Decision Transformer* serves as the backbone of the recommendation policy, modeling user-item trajectories as sequences of return-to-go (RTG), states, and actions (g_t, s_t, a_t) . These temporal dependencies can be captured by any causal Transformer architecture; in our implementation, we employ GPT-2 as the causal Transformer backbone to model sequential dependencies and maintain autoregressive consistency. During the offline pretraining stage,

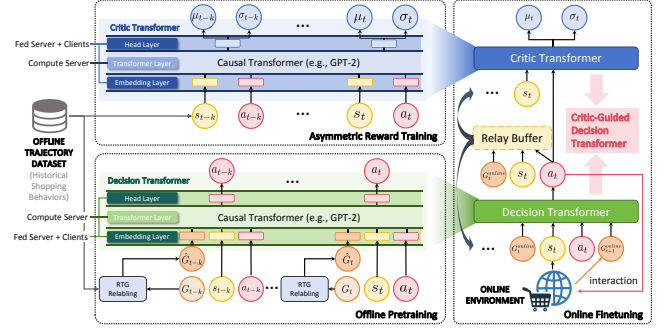


Figure 2: Architecture of FeedGuard

the DT learns to predict actions conditioned on past states and RTG values, approximating the optimal policy π_{DT} .

To enhance value estimation, FeedGuard introduces a *Critic Transformer* that adopts the same GPT-2 architecture as the Decision Transformer but is trained independently to model the return distribution $p(g_t | s_t, a_t)$, parameterized by mean μ_t and variance σ_t . This critic provides long-term value guidance to the DT, forming a *critic-guided Decision Transformer*. Unlike traditional DT-based recommendation systems that rely solely on observed RTGs, this design enables proactive adjustment of predicted returns, leading to more stable and farsighted policy updates.

The *Replay Buffer* is designed to stabilize online training and improve sample efficiency under continuously changing user behaviors. In real-world recommendation environments, user interaction patterns evolve over time, requiring continual fine-tuning of the policy. However, direct step-by-step updates based on instantaneous feedback can lead to training instability and overfitting to suboptimal trajectories. To mitigate this issue, FeedGuard maintains a replay buffer that not only collects recent user-item interactions from the online environment but also performs trajectory relabeling by stitching together high-reward sub-trajectories. These curated sequences are periodically replayed to jointly update both the Decision and Critic Transformers, enabling stable and sample-efficient online adaptation to dynamic user preferences.

Federated Split Execution. To enforce privacy-preserving computation across all training stages, FeedGuard employs a unified split execution scheme for both *Decision Transformer* and *Critic Transformer* within a federated learning paradigm, where model parameters and computations are distributed across three modules based on their sensitivity to privacy and computational load.

Specifically, the *clients* retain the embedding and head layers of both transformers to process raw user interactions and compute gradients locally, ensuring that personal features and behavioral representations never leave the device. The *compute server* hosts the shared transformer layers, performing forward and backward propagation for representation learning under secure split learning. Meanwhile, the *federated server* coordinates global aggregation by averaging client-side updates with differential privacy noise, synchronizing parameters of both the Decision and Critic Transformers across clients without accessing raw gradients or latent embeddings. This consistent split design enables both models to benefit from federated coordination while ensuring that sensitive feedback and model updates remain securely confined to local devices.

3.3 Training Lifecycle of FeedGuard

In this section, we introduce FeedGuard, which organizes DTRS learning into a three-stage feedback lifecycle: *offline pretraining*, *asymmetric reward training*, and *online fine-tuning*.

Offline Pretraining. This stage operates as follows. Each client first synchronizes with the latest global parameters and performs a forward pass through its *local embedding* layers on historical interaction trajectories to obtain latent representations E_k . These representations are privatized on-device (e.g., Gaussian mechanism) to yield $\tilde{E}_k$ and sent to the *compute server*, which hosts the transformer layers. The compute server produces intermediate states $H_k = \text{Transformer}(\tilde{E}_k)$ and returns H_k to the client. The client then applies its *local head* layers to obtain final outputs and compute the loss. For backpropagation, the client sends $\partial\mathcal{L}/\partial H_k$ to the compute server; the server backpropagates through the transformer to obtain $\partial\mathcal{L}/\partial\tilde{E}_k$ and returns it, after which the client completes gradients for its embedding layers and updates local parameters. Per training round, the *fed server* performs *synchronous*, sample-size-weighted aggregation (FedAvg) of differentially-private server-side gradients to update global parameters, $W^{t+1} = \sum_{k=1}^K \frac{n_k}{N} W_k^t$ with $N = \sum_k n_k$, and broadcasts the updated model to all clients. Throughout, all cross-party transmissions (latent representations and gradients) are protected by differential privacy.

Given this procedure, the offline objective maximizes the log-likelihood of trajectories sampled from the offline dataset $\mathcal{D}$:

$$J(\theta) = \frac{1}{K} \mathbb{E}_{(a,s,g) \sim \mathcal{T}} \left[- \sum_{k=1}^K \log \pi_{\theta}(a_k | s_{k-K:k}, g_{k-K:k}) \right], \quad (4)$$

where $\mathcal{T}$ denotes the distribution over sampled trajectories.

Asymmetric Reward Training. Operationally, this stage mirrors *offline pretraining*: each client uses local interactions to produce embeddings with its *embedding* layers, privatizes these latent representations on-device, and sends them to the *compute server* for transformer-layer computation; the returned hidden states are passed through the client *head* layers to compute the loss, gradients are sent back to the compute server for transformer backpropagation, and the *fed server* aggregates client updates into a global critic model. All cross-party transmissions (latent representations and gradients) are protected by differential privacy. Given this pipeline, we introduce a critic to improve long-term value estimation. In recommendation, selecting actions toward a target return requires modeling the conditional action distribution; by Bayes' rule, $p(a_t | R_t, s_t) \propto p(a_t | s_t) p(R_t | s_t, a_t)$, highlighting the need to estimate $p(R_t | s_t, a_t)$. We approximate this distribution with a Gaussian critic $Q_{\phi}(R_t | \tau_{0:t-1}, s_t, a_t)$ [33] and optimize it using an asymmetric negative log-likelihood:

$$L_Q(\phi) = -|\tau_c - I(u > 0)| \cdot \log Q_{\phi}(R_t | \tau_{0:t-1}, s_t, a_t), \quad (5)$$

where $u = \frac{R_t - \mu_t}{\sigma_t}$ and (μ_t, σ_t) are the critic's predicted mean and variance. The asymmetry parameter τ_c biases the critic toward high-reward samples when $\tau_c > 0.5$ and toward suboptimal ones when $\tau_c < 0.5$, yielding robust long-term value modeling under the same split-federated workflow.

Online Fine-tuning. Clients interact with users in real time, generating recommendations with locally adapted critic models. New interactions (s_t, a_t, r_t, s_{t+1}) are appended to a local replay

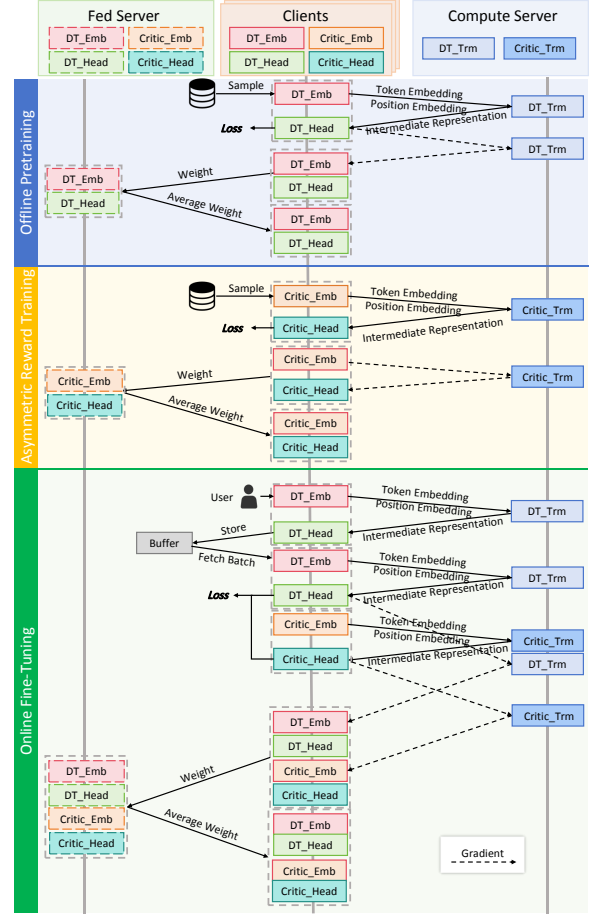


Figure 3: The Workflow of FeedGuard

buffer and periodically sampled for training. For each minibatch, the client runs its *embedding* layers to obtain latent representations, privatizes them on-device, and sends the latents to the *compute server* for transformer-layer computation; the returned hidden states are used for client-side refinement via local *head* layers and loss computation. The critic evaluates feedback locally and guides action selection by

$$a_t = \arg \max_a \left[Q_{\phi}(s_t, a) + \pi_{DT}(a | s_{t-K:t}, g_{t-K:t}) \right], \quad (6)$$

after which the environment returns (s_{t+1}, r_t) and the return-to-go is updated as $g_{t+1} = g_t - r_t$, enabling autoregressive rollout. Gradients are perturbed on-device and *asynchronously aggregated* by the fed server, enabling timely global updates with privacy protection during continuous fine-tuning. To encourage exploration and prevent premature convergence, we optimize with Max-Entropy regularization [38] alongside critic guidance:

$$\min_{\theta} J(\theta) - \lambda H_{\theta}^T[a | s, g] + \alpha \cdot L_Q(\phi), \quad (7)$$

where λ and α control the trade-off among exploration, exploitation, and critic alignment. The trajectory-level entropy is

$$H_{\theta}^T[a | s, g] = \frac{1}{K} \mathbb{E}_{(s,g) \sim \mathcal{T}} \left[\sum_{k=1}^K H[\pi_{\theta}(a_k | s_{k-K:k}, g_{k-K:k})] \right], \quad (8)$$

which promotes diversity in action sampling. With a replay buffer $\mathcal{T}_{\text{replay}}$ mixing recent and historical interactions, the full objective mirrors offline training with added entropy and critic terms:

$$J(\theta) = \frac{1}{K} \mathbb{E}_{(a,s,g) \sim \mathcal{T}_{\text{replay}}} \left[- \sum_{k=1}^K \log \pi_{\theta}(a_k | s_{k-K:k}, g_{k-K:k}) - \lambda H_{\theta}^T[a | s, g] + \alpha \cdot L_Q(\phi) \right]. \quad (9)$$

This online stage stabilizes value estimates, curbs overestimation on out-of-distribution actions, and enables rapid, privacy-preserving adaptation in non-stationary recommendation environments.

3.4 Privacy Budget Allocation

FeedGuard enforces differential privacy (DP) at multiple stages of training. Since each phase involves repeated transmission of privatized data, the overall privacy loss accumulates across rounds and stages. To ensure an end-to-end (ϵ, δ) -DP guarantee, we need to determine how to allocate the total privacy budget across structurally distinct training stages.

Let T_i denote the number of rounds in stage $i \in \{\text{off}, \text{rew}, \text{on}\}$, with a shared sampling rate q and a common noise scale σ . We define per-stage privacy allocation as $\epsilon_i = \lambda_i \cdot \epsilon$, and aim to determine the allocation weights λ_i and the minimum required noise level to satisfy the global privacy guarantee. Assuming a uniform noise scale simplifies system design, allowing consistent DP mechanisms across stages without stage-specific tuning.

THEOREM 1 (COMMON NOISE BOUND UNDER LIFECYCLE PRIVACY BUDGET ALLOCATION). *Let the training stages satisfy the relationship $\sqrt{T_{\text{off}}} : \sqrt{T_{\text{rew}}} : \sqrt{T_{\text{on}}} = 1 : 1 : \sqrt{2}$, derived from the number of differential privacy events triggered per round in each stage. Assume each stage employs the same Gaussian noise scale σ with per-round sampling rate q . Then, to achieve an overall (ϵ, δ) -DP guarantee under the Moments Accountant framework, the minimum noise scale σ for each stage i must satisfy:*

$$\sigma > c \cdot \frac{q \cdot \sqrt{T_i \log(1/\delta)}}{\lambda_i \cdot \epsilon}, \quad (10)$$

where the allocation weights are $\lambda_{\text{off}} = \lambda_{\text{rew}} = \frac{1}{2+\sqrt{2}}$, $\lambda_{\text{on}} = \frac{\sqrt{2}}{2+\sqrt{2}}$.

This formulation offers a principled strategy to stage-aware noise calibration within a unified privacy framework. A formal proof is provided in Appendix B.

4 Experiment

In this section, we present extensive experiments to answer the following questions: **RQ1**: How does FeedGuard perform against baseline models in offline and online recommendation? **RQ2**: How does each training stage in FeedGuard contribute to recommendation performance? **RQ3**: How does privacy protection influence the effectiveness of FeedGuard? **RQ4**: How does the privacy budget affect the performance of FeedGuard?

Table 1: Statistics of the datasets

Dataset	MovieLens-1M	Amazon CD	MovieLens-20M	Netflix
# Users	6,040	1,578,597	138,493	480,189
# Items	3,952	486,360	27,278	17,770
# Interactions	1,000,209	3,749,004	20,000,263	100,480,507
Avg. of actions / user	165.60	2.37	144.41	209.25
Avg. of actions / item	269.89	7.70	747.84	5654.50
Density	4.4664%	0.0005%	0.5400%	1.1775%

4.1 Experimental Settings

Dataset and Simulation Environment. We assess the performance of our proposed FeedGuard utilizing both public offline datasets and an online simulation setup. Dataset statistics are summarized in Table 1 and implementation details including data processing and model configurations are provided in the Appendix C.

Comparison Methods. We compare FeedGuard with three categories of baselines: (1) *Transformer-based sequential recommendation methods*, including GRU4Rec [14], SASRec [16], and BERT4Rec [27]; (2) *Classic online RL-based recommendation methods*, such as DDPG [19], PPO [25], and TD3 [10], which are adapted to recommendation scenarios by leveraging user interactions to continuously optimize policies; (3) *Offline RLRS methods using decision transformers*. In our online experiments, we include DT [5], CDT4Rec [30], and EDT4Rec [6]. DT utilizes historical trajectories of state-action-reward triples for autoregressive decision-making. CDT4Rec extends this by incorporating a causal layer, enhancing the ability to capture user preferences during recommendation. EDT4Rec further introduces an improved exploration strategy to optimize recommendations by effectively balancing exploration and exploitation. In our offline experiments, we also adapt DT4Rec [37] and DT4IER [21], methods originally proposed for long-term user engagement tasks (e.g., user retention). DT4Rec is a sequential recommendation method that uses decision transformers to take advantage of both reinforcement learning and supervised learning for improved performance. DT4IER employs a high-dimensional encoder with contrastive learning to enhance prediction performance in the multi-reward recommendation task.

Metrics. The performance of our model is evaluated through different metrics in both offline and online environments. For offline recommendation, We use two widely adopted metrics [16, 27] in sequential recommendation: **nDCG@K** measures the normalized cumulative gain of the top-K recommendations, considering both the relevance of items and their positions in the ranking; **HR@K** indicates the probability that ground-truth items are present in the top-K recommendations. Note that HR@K is similar to Recall@K and is directly related to Precision@K, given that each user has only one ground truth item.

For online recommendation environments, we adopt click-through rate (CTR) as the effectiveness metric, calculated by formula: $\text{CTR} = \frac{\text{episode_return}}{\text{episode_length} \times \text{maximum_reward}}$, where maximum reward denotes the highest possible reward achievable in a single step of an episode.

4.2 Overall Comparison (RQ1)

Result 1: Overall Performance in Offline Recommendation. To address RQ1, we compare transformer-based and decision transformer-based sequential recommenders on offline tasks. As shown

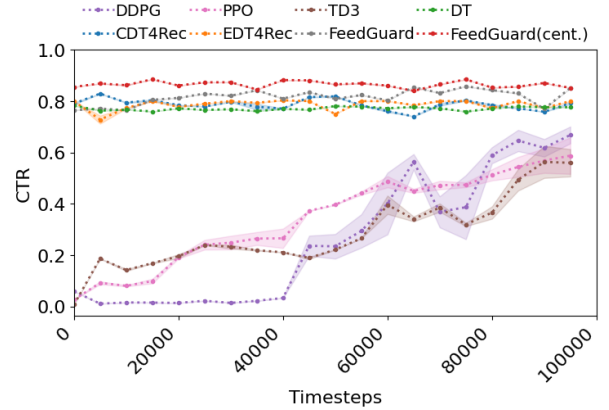
Table 2: Comparison of different models on various datasets in terms of nDCG and HR.

Dataset	MovieLens-1M				Amazon CD			
	nDCG@10	nDCG@20	HR@10	HR@20	nDCG@10	nDCG@20	HR@10	HR@20
<i>Transformer-based Methods</i>								
GRU4Rec	47.97 $\pm$ 0.43	50.40 $\pm$ 0.41	68.42 $\pm$ 0.38	78.01 $\pm$ 0.31	32.73 $\pm$ 1.51	35.74 $\pm$ 1.52	49.68 $\pm$ 2.00	61.58 $\pm$ 2.16
SASRec	45.77 $\pm$ 0.38	48.25 $\pm$ 0.34	67.18 $\pm$ 0.27	76.98 $\pm$ 0.14	<u>34.66 $\pm$ 4.29</u>	<u>37.30 $\pm$ 4.31</u>	50.28 $\pm$ 5.21	60.76 $\pm$ 5.34
BERT4Rec	47.67 $\pm$ 0.37	50.08 $\pm$ 0.37	68.69 $\pm$ 0.43	78.16 $\pm$ 0.37	25.22 $\pm$ 1.12	28.18 $\pm$ 1.03	40.14 $\pm$ 1.17	51.91 $\pm$ 0.95
<i>Decision Transformer-based Methods</i>								
CDT4Rec	49.01 $\pm$ 0.25	52.55 $\pm$ 0.26	69.17 $\pm$ 0.28	80.01 $\pm$ 0.26	33.01 $\pm$ 0.42	36.06 $\pm$ 0.45	50.13 $\pm$ 0.53	62.01 $\pm$ 0.58
EDT4Rec	48.73 $\pm$ 0.24	52.38 $\pm$ 0.27	70.05 $\pm$ 0.26	79.64 $\pm$ 0.25	33.18 $\pm$ 0.41	36.24 $\pm$ 0.44	50.36 $\pm$ 0.52	62.32 $\pm$ 0.57
DT4Rec	48.33 $\pm$ 0.25	51.35 $\pm$ 0.24	65.62 $\pm$ 0.16	77.53 $\pm$ 0.22	31.03 $\pm$ 0.26	33.93 $\pm$ 0.32	41.49 $\pm$ 0.24	53.02 $\pm$ 0.51
DT4IER	48.45 $\pm$ 0.07	51.49 $\pm$ 0.03	65.99 $\pm$ 0.22	78.06 $\pm$ 0.13	30.80 $\pm$ 0.06	33.60 $\pm$ 0.06	41.25 $\pm$ 0.06	52.42 $\pm$ 0.07
<i>Ours</i>								
FeedGuard (cent.)	49.73 $\pm$ 0.25	53.45 $\pm$ 0.26	70.28 $\pm$ 0.28	81.45 $\pm$ 0.26	34.47 $\pm$ 0.42	37.52 $\pm$ 0.45	50.89 $\pm$ 0.53	63.12 $\pm$ 0.58
FeedGuard	48.85 $\pm$ 0.25	52.82 $\pm$ 0.26	69.45 $\pm$ 0.28	79.68 $\pm$ 0.26	32.96 $\pm$ 0.42	36.25 $\pm$ 0.45	50.21 $\pm$ 0.53	61.78 $\pm$ 0.58
Improvement	+1.47%	+1.71%	+0.33%	+1.80%	-0.55%	+0.59%	+1.05%	+1.28%
<i>MovieLens-20M</i>								
Dataset	MovieLens-20M				Netflix			
	nDCG@10	nDCG@20	HR@10	HR@20	nDCG@10	nDCG@20	HR@10	HR@20
<i>Transformer-based Methods</i>								
GRU4Rec	45.07 $\pm$ 1.37	48.12 $\pm$ 1.28	66.47 $\pm$ 1.49	78.53 $\pm$ 1.10	36.19 $\pm$ 0.66	39.36 $\pm$ 0.66	53.92 $\pm$ 0.87	66.47 $\pm$ 0.87
SASRec	45.35 $\pm$ 1.30	48.36 $\pm$ 1.25	66.80 $\pm$ 1.16	78.68 $\pm$ 0.98	32.90 $\pm$ 0.22	36.29 $\pm$ 0.21	51.31 $\pm$ 0.17	64.78 $\pm$ 0.32
BERT4Rec	46.91 $\pm$ 0.70	49.83 $\pm$ 0.67	68.01 $\pm$ 0.75	79.52 $\pm$ 0.64	34.34 $\pm$ 0.20	37.53 $\pm$ 0.18	52.03 $\pm$ 0.22	64.67 $\pm$ 0.12
<i>Decision Transformer-based Methods</i>								
CDT4Rec	47.65 $\pm$ 0.70	50.68 $\pm$ 0.67	69.22 $\pm$ 0.75	80.89 $\pm$ 0.64	39.35 $\pm$ 0.05	42.51 $\pm$ 0.08	54.49 $\pm$ 0.09	67.01 $\pm$ 0.17
EDT4Rec	47.58 $\pm$ 0.48	50.72 $\pm$ 0.45	69.30 $\pm$ 0.52	80.76 $\pm$ 0.43	39.47 $\pm$ 0.05	42.61 $\pm$ 0.08	54.66 $\pm$ 0.09	67.19 $\pm$ 0.17
DT4Rec	49.45 $\pm$ 0.10	52.47 $\pm$ 0.11	66.52 $\pm$ 0.12	78.48 $\pm$ 0.16	39.03 $\pm$ 0.05	42.15 $\pm$ 0.08	54.03 $\pm$ 0.09	66.42 $\pm$ 0.17
DT4IER	49.40 $\pm$ 0.02	52.43 $\pm$ 0.03	66.43 $\pm$ 0.02	78.46 $\pm$ 0.07	38.87 $\pm$ 0.06	41.98 $\pm$ 0.07	53.76 $\pm$ 0.08	66.59 $\pm$ 0.15
<i>Ours</i>								
FeedGuard (cent.)	50.28 $\pm$ 0.02	53.28 $\pm$ 0.03	70.48 $\pm$ 0.50	82.09 $\pm$ 0.41	40.18 $\pm$ 0.05	43.35 $\pm$ 0.08	55.54 $\pm$ 0.09	68.43 $\pm$ 0.17
FeedGuard	49.53 $\pm$ 0.02	52.25 $\pm$ 0.03	69.48 $\pm$ 0.52	80.45 $\pm$ 0.43	39.63 $\pm$ 0.05	42.49 $\pm$ 0.08	54.78 $\pm$ 0.09	67.03 $\pm$ 0.17
Improvement	+1.68%	+1.54%	+1.70%	+1.48%	+1.80%	+1.74%	+1.61%	+1.85%

in Table 2, FeedGuard (cent.) (i.e., the centralized variant without federated privacy protection) achieves the best overall performance, improving by 6.36% on average over all baselines and 1.78% over the best decision transformer-based methods. This result highlights the effectiveness of critic guidance for long-term value estimation and trajectory-level optimization. Moreover, the privacy-preserving version of FeedGuard remains competitive with decision transformer-based methods and consistently outperforms transformer-based baselines, with only modest degradation due to privacy protection, indicating that user feedback can be effectively safeguarded without significant compromise in recommendation quality.

Result 2: CTR Performance in Online Recommendation.

To address RQ1, we compare FeedGuard with online RL-based recommendation methods. Figure 4 presents the CTR performance over different interaction rounds. Overall, DTRS-based methods generally perform better, benefiting from stronger sequence modeling capabilities. Notably, DTRS-based methods exhibit high and stable CTR from the very beginning of interaction, indicating the effectiveness of offline pretraining. Among them, FeedGuard (cent.) attains the best average CTR, outperforming the strongest baseline by 9.89%, demonstrating improved sample efficiency through critic-guided online adaptation. Notably, the federated variant retains comparable performance, with only a 3.43% CTR drop under privacy constraints, indicating that asynchronous federated fine-tuning effectively preserves recommendation quality despite communication delays and DP noise.

**Figure 4: Overall comparison**

4.3 Ablation Experiment (RQ2 and RQ3)

To investigate RQ2 and RQ3, we evaluate FeedGuard under different combinations of training stages and privacy mechanisms. Specifically, we consider three training configurations: (1) offline pretraining only, (2) online fine-tuning only, and (3) full-stage configuration. Each is tested under three privacy settings: (a) centralized, (b) federated learning (FL), and (c) federated learning with differential privacy (FL+DP). We report results across all stage-privacy combinations and summarize two key results below.

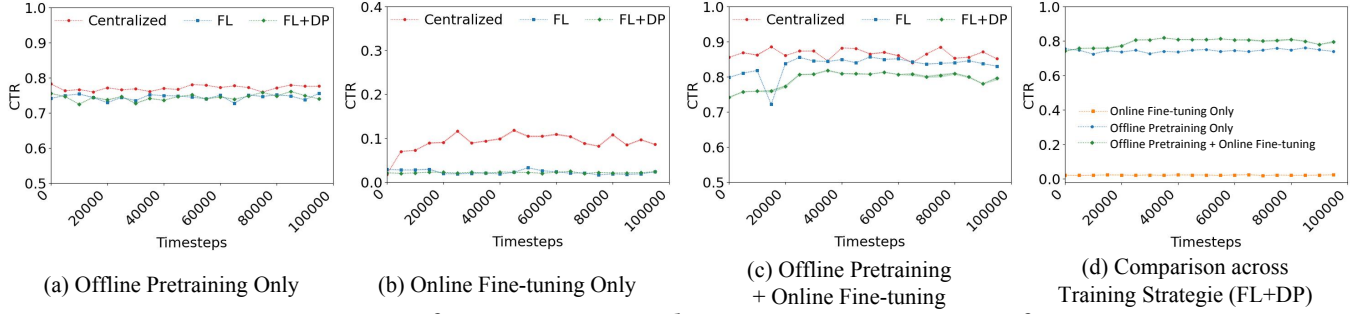
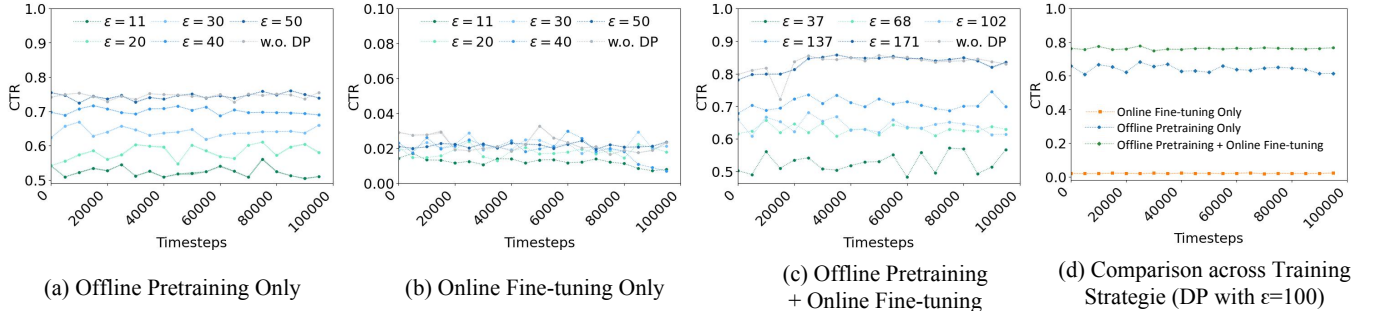


Figure 5: Impact of Training Strategy and Privacy Protection on CTR Performance

Figure 6: Impact of Privacy Budget ϵ on CTR Performance (Note that the ϵ represents the total budget consumed across all clients over the entire training lifecycle.)

Result 3: Effect of Training Stages. We observe that training with both offline pretraining and online fine-tuning yields much higher CTR than using either stage alone. In the FL+DP setting (cf. Figure 5 (d)), this full-stage configuration yields a 10.10% improvement over offline pretraining only training. The gain primarily stems from the complementarity between critic-guided offline initialization and online asynchronous adaptation, where the critic provides stable value guidance, while fine-tuning captures evolving user behaviors. In contrast, the online fine-tuning only variant exhibits almost no learning progress, with CTR values remaining below 0.1 across all privacy configurations. These results highlight the importance of high-quality offline trajectories for stabilizing and bootstrapping training in privacy-preserving recommendation.

Result 4: Effect of Privacy Protection. We observe moderate CTR drops when applying FL or DP in both offline pretraining only (Figure 5 (a)) and full-stage settings (Figure 5 (c)). Due to the extremely poor performance of the online fine-tuning only variant (Figure 5 (b)), the impact of privacy mechanisms in this setting is negligible and thus omitted. Among the remaining settings, the full-stage configuration shows the largest performance drop under FL+DP, but the degradation remains within an acceptable range. Notably, the small gap between FL and FL+DP indicates that the multi-stage DP design incurs only minimal additional loss, enabling stronger privacy without sacrificing recommendation quality.

4.4 Hyperparameters Study (RQ4)

Result 5: Effect of Privacy Budget. To address RQ4, we examine the effect of varying the privacy budget ϵ across different training stages. As shown in Figure 6 (a–c), adjusting ϵ enables a clear privacy–performance trade-off across all stage configurations. In

the full-stage setting (Figure 6 (d)), budget splitting across stages introduces stronger noise per stage, leading to a relatively larger performance drop than offline pretraining only. Nevertheless, with a stage-aware budget allocation strategy, FeedGuard still achieves competitive performance under tight privacy constraints. These findings validate the effectiveness of our multi-stage DP design in preserving utility while ensuring end-to-end privacy protection.

5 Conclusions and Future Work

In this paper, we propose FeedGuard, a critic-guided reinforcement learning framework with privacy-preserving feedback for recommendation. Experiments on public datasets and VirtualTB platform confirmed that FeedGuard performs strongly in both offline and on-line settings, while maintaining rigorous privacy guarantees with minimal degradation. Despite these promising results, FeedGuard has two limitations: robustness under extreme client data imbalance remains unexplored, and it assumes stable client–server connectivity, which may not hold in volatile networks. In future work, we plan to extend FeedGuard with robust aggregation methods for heterogeneous client data and fault-tolerant mechanisms for unreliable links. We also aim to study how privacy-preserving learning interacts with broader concerns such as fairness and transparency in recommendation systems.

Acknowledgments

This work was supported in part by the “Pioneer” and “Leading Goose” R&D Program of Zhejiang (No. 2024C01212), and Hangzhou Key R&D Program (Project No.2024SZD1A33).

References

- [1] Martin Abadi, Andy Chu, Ian Goodfellow, H Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. 2016. Deep learning with differential privacy. In *Proc. of the ACM SIGSAC*. 308–318.
- [2] David Brandfonbrener, Alberto Bietti, Jacob Buckman, Romain Laroche, and Joan Bruna. 2022. When does return-conditioned supervised learning work for offline reinforcement learning?. In *Proc. of NeurIPS*.
- [3] Han Cai, Kan Ren, Weinan Zhang, Kleanthis Malialis, Jun Wang, Yong Yu, and Defeng Guo. 2017. Real-time bidding by reinforcement learning in display advertising. In *Proc. of WSDM*. 661–670.
- [4] Qingpeng Cai, Aris Filos-Ratsikas, Pingzhong Tang, and Yiwei Zhang. 2018. Reinforcement Mechanism Design for e-commerce. In *Proc. of WWW*. 1339–1348.
- [5] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Misha Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. 2021. Decision transformer: Reinforcement learning via sequence modeling. *Advances in neural information processing systems* 34 (2021), 15084–15097.
- [6] Xiaocong Chen, Siyu Wang, and Lina Yao. 2024. Maximum-Entropy Regularized Decision Transformer with Reward Relabelling for Dynamic Recommendation. In *Proc. of KDD*. 376–384.
- [7] Romain Deffayet, Thibaut Thonet, Jean-Michel Renders, and Maarten De Rijke. 2023. Offline evaluation for reinforcement learning-based recommendation: a critical issue and some alternatives. In *Proc. of SIGIR*, Vol. 56. ACM New York, NY, USA, 1–14.
- [8] Cynthia Dwork, Krishnamurthy Kenthapadi, Frank McSherry, Ilya Mironov, and Moni Naor. 2006. Our data, ourselves: Privacy via distributed noise generation. In *Proc. of the EUROCRYPT*. Springer, 486–503.
- [9] Cynthia Dwork, Aaron Roth, et al. 2014. The algorithmic foundations of differential privacy. *Foundations and Trends® in Theoretical Computer Science* 9, 3–4 (2014), 211–407.
- [10] Scott Fujimoto, Herke Hoof, and David Meger. 2018. Addressing function approximation error in actor-critic methods. In *Proc. of ICML*. PMLR, 1587–1596.
- [11] Chongming Gao, Kexin Huang, Jiawei Chen, Yuan Zhang, Biao Li, Peng Jiang, Shiqi Wang, Zhong Zhang, and Xiangnan He. 2023. Alleviating matthew effect of offline reinforcement learning in interactive recommendation. In *Proc. of SIGIR*. 238–248.
- [12] Elizabeth Liz Harding, Jarno J Vanto, Reece Clark, L Hannah Ji, and Sara C Ainsworth. 2019. Understanding the scope and impact of the california consumer privacy act of 2018. *Journal of Data Protection & Privacy* 2, 3 (2019), 234–253.
- [13] Liam Hebert, Lukasz Golab, Pascal Poupart, and Robin Cohen. 2022. Fedformer: Contextual federation with attention in reinforcement learning. *arXiv preprint arXiv:2205.13697* (2022).
- [14] B Hidasi. 2015. Session-based Recommendations with Recurrent Neural Networks. *arXiv preprint arXiv:1511.06939* (2015).
- [15] Yiqiu Hu, Yun Hua, Wenyan Liu, and Jun Zhu. 2021. Reward shaping based federated reinforcement learning. *IEEE Access* 9 (2021), 67259–67267.
- [16] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *Proc. of ICDM*. IEEE, 197–206.
- [17] Zitao Li, Bolin Ding, Ce Zhang, Ninghui Li, and Jingren Zhou. 2021. Federated matrix factorization with privacy guarantee. *Proc. VLDB Endowment* 15, 4 (2021).
- [18] Xinting Liao, Weiming Liu, Xiaolin Zheng, Binhui Yao, and Chaochao Chen. 2023. Ppgendr: A stable and robust framework for privacy-preserving cross-domain recommendation. In *Proc. of AAAI*, Vol. 37. 4453–4461.
- [19] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2015. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971* (2015).
- [20] Zhaohao Lin, Weike Pan, Qiang Yang, and Zhong Ming. 2022. A generic federated recommendation framework via fake marks and secret sharing. *ACM Transactions on Information Systems* 41, 2 (2022), 1–37.
- [21] Ziru Liu, Shuchang Liu, Zijian Zhang, Qingpeng Cai, Xiangyu Zhao, Kesen Zhao, Lantao Hu, Peng Jiang, and Kun Gai. 2024. Sequential recommendation for optimizing both immediate feedback and long-term retention. In *Proc. of SIGIR*. 1872–1882.
- [22] Omar Moling, Linas Baltrunas, and Francesco Ricci. 2012. Optimal radio channel recommendations with explicit and implicit feedback. In *Proc. of RecSys*. 75–82.
- [23] Jiaju Qi, Qihao Zhou, Lei Lei, and Kan Zheng. 2021. Federated reinforcement learning: Techniques, applications, and open challenges. *arXiv preprint arXiv:2108.11887* (2021).
- [24] Desik Rengarajan, Nitin Raghothaman, Dileep Kalathil, and Srinivas Shakkottai. 2024. Federated ensemble-directed offline reinforcement learning. *Advances in Neural Information Processing Systems* 37 (2024), 6154–6179.
- [25] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
- [26] Jing-Cheng Shi, Yang Yu, Qing Da, Shi-Yong Chen, and An-Xiang Zeng. 2019. Virtual-taobao: Virtualizing real-world online retail environment for reinforcement learning. In *Proc. of the AAAI*, Vol. 33. 4902–4909.
- [27] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proc. of CIKM*. 1441–1450.
- [28] Chandra Thapa, Pathum Chamikara Mahawaga Arachchige, Seyit Camtepe, and Lichao Sun. 2022. Splitfed: When federated learning meets split learning. In *Proc. of the AAAI*, Vol. 36. 8485–8493.
- [29] Paul Voigt and Axel Von dem Bussche. 2017. The eu general data protection regulation (gdpr). *A Practical Guide, 1st Ed., Cham: Springer International Publishing* 10, 3152676 (2017), 10–5555.
- [30] Siyu Wang, Xiaocong Chen, Dietmar Jannach, and Lina Yao. 2023. Causal decision transformer for recommender systems via offline reinforcement learning. In *Proc. of SIGIR*. 1599–1608.
- [31] Yong Wang, Mingxing Gao, Xun Ran, Jun Ma, and Leo Yu Zhang. 2023. An improved matrix factorization with local differential privacy based on piecewise mechanism for recommendation systems. *Expert Systems with Applications* 216 (2023), 119457.
- [32] Yuyan Wang, Mohit Sharma, Can Xu, Sriraj Badam, Qian Sun, Lee Richardson, Lisa Chung, Ed H Chi, and Minmin Chen. 2022. Surrogate for long-term user experience in recommender systems. In *Proc. of KDD*. 4100–4109.
- [33] Yuanfu Wang, Chao Yang, Ying Wen, Yu Liu, and Yu Qiao. 2024. Critic-guided decision transformer for offline reinforcement learning. In *Proc. of the AAAI*, Vol. 38. 15706–15714.
- [34] Zhijie Xie and Shenghui Song. 2023. FedKL: Tackling data heterogeneity in federated reinforcement learning by penalizing KL divergence. *IEEE Journal on Selected Areas in Communications* 41, 4 (2023), 1227–1242.
- [35] Enyue Yang, Yunfeng Huang, Feng Liang, Weike Pan, and Zhong Ming. 2021. FCMF: Federated collective matrix factorization for heterogeneous collaborative filtering. *Knowledge-Based Systems* 220 (2021), 106946.
- [36] Wei Yuan, Chaoqun Yang, Guanhua Ye, Tong Chen, Nguyen Quoc Viet Hung, and Hongzhi Yin. 2024. FELLAS: Enhancing Federated Sequential Recommendation with LLM as External Services. *ACM Transactions on Information Systems* (2024).
- [37] Kesen Zhao, Lixin Zou, Xiangyu Zhao, Maolin Wang, and Dawei Yin. 2023. User retention-oriented recommendation with decision transformer. In *Proc. of WWW*. 1141–1149.
- [38] Qingqing Zheng, Amy Zhang, and Aditya Grover. 2022. Online decision transformer. In *Proc. of ICML*. PMLR, 27042–27059.
- [39] Xiaoyao Zheng, Manping Guan, Xianmin Jia, Liangmin Guo, and Yonglong Luo. 2022. A matrix factorization recommendation system-based local differential privacy for protecting users' sensitive data. *IEEE Transactions on Computational Social Systems* 10, 3 (2022), 1189–1198.
- [40] Doudou Zhou, Yufeng Zhang, Aaron Sonabend-W, Zhaoran Wang, Junwei Lu, and Tianxi Cai. 2024. Federated offline reinforcement learning. *J. Amer. Statist. Assoc.* 119, 548 (2024), 3152–3163.

A Algorithms in FeedGuard

To clarify our implementation, we summarize the FeedGuard training workflow in pseudocode, divided into three components: the compute server, the federated server, and the client, each responsible for different stages within the split federated learning pipeline.

Algorithm 1 outlines the compute server's operations, which execute forward and backward passes of the transformer layers for both the Decision Transformer and the Critic Transformer. It receives differentially private intermediate representations from clients, computes hidden representations, and returns gradients for local updates.

Algorithm 1 Compute Server Procedure

Notation: $\tilde{E}_k, \tilde{C}_k$: DP-protected embeddings received from client-side embedding layers as inputs to the server-side Decision Transformer and Critic Transformer, respectively.

H_k, G_k : hidden representations from server-side Decision Transformer and Critic Transformer.

$\frac{\partial \mathcal{L}_k}{\partial H_k}, \frac{\partial \mathcal{L}_k}{\partial G_k}$: DP-protected gradients from client-side head layers.

$\frac{\partial \mathcal{L}_k}{\partial E_k}, \frac{\partial \mathcal{L}_k}{\partial C_k}$: gradients w.r.t. client embeddings to be sent back to clients.

η_{DT}, η_{Critic} : server learning rate for Decision Transformer and Critic Transformer, respectively.

```

1: Input: time  $t$ , client set  $\mathcal{S}_t$ , weights  $W_t^{DT}, W_t^{Critic}$ 
2: for all client  $k \in \mathcal{S}_t$  do
3:   // Forward pass through server-side transformer layers
4:   Receive  $\tilde{E}_k$  from client,  $H_k \leftarrow \text{Transformer}_{DT}(\tilde{E}_k; W_t^{DT})$ 
5:   Send  $H_k$  back to client
6:   Receive  $\tilde{C}_k, G_k \leftarrow \text{Transformer}_{Critic}(\tilde{C}_k; W_t^{Critic})$ 
7:   Send  $G_k$  back to client
8:   // Backward pass to compute gradients w.r.t. client inputs
9:   Receive  $\frac{\partial \mathcal{L}_k}{\partial H_k}, \frac{\partial \mathcal{L}_k}{\partial G_k}$  from client
10:   $\frac{\partial \mathcal{L}_k}{\partial W_t^{Critic}} \leftarrow \text{Backprop}(\frac{\partial \mathcal{L}_k}{\partial G_k}, G_k, W_t^{Critic})$ 
11:   $\frac{\partial \mathcal{L}_k}{\partial W_t^{DT}} \leftarrow \text{Backprop}(\frac{\partial \mathcal{L}_k}{\partial H_k}, H_k, W_t^{DT})$ 
12:   $\frac{\partial \mathcal{L}_k}{\partial C_k} \leftarrow \frac{\partial \mathcal{L}_k}{\partial G_k} \cdot \frac{\partial G_k}{\partial C_k}, \frac{\partial \mathcal{L}_k}{\partial E_k} \leftarrow \frac{\partial \mathcal{L}_k}{\partial H_k} \cdot \frac{\partial H_k}{\partial E_k}$ 
13:  Send  $\frac{\partial \mathcal{L}_k}{\partial E_k}, \frac{\partial \mathcal{L}_k}{\partial C_k}$  back to client
14:   $W_t^{DT} \leftarrow W_t^{DT} - \eta_{DT} \cdot \frac{1}{|\mathcal{S}_t|} \sum_{k \in \mathcal{S}_t} \frac{\partial \mathcal{L}_k}{\partial W_t^{DT}}$ 
15:   $W_t^{Critic} \leftarrow W_t^{Critic} - \eta_{Critic} \cdot \frac{1}{|\mathcal{S}_t|} \sum_{k \in \mathcal{S}_t} \frac{\partial \mathcal{L}_k}{\partial W_t^{Critic}}$ 
16: end for

```

Algorithm 2 summarizes the federated server's aggregation process, which applies synchronous FedAvg in offline pretraining and asynchronous updates online to efficiently synchronize models across heterogeneous clients.

Algorithm 3 specifies the client-side procedure. Each client locally trains the embedding and head layers of the Decision Transformer and Critic network using private user data, and communicates with compute and federated servers by exchanging privacy-preserving representations and differentially private model updates.

Algorithm 2 Fed Server Procedure with Synchronous and Asynchronous Aggregation

Notation: $W_t, W_{t,k}$: global client weights at round t and local client weights update from client k .

mode stage $\in \{\text{OFFLINE PRETRAINING}, \text{REWARD TRAINING}, \text{ONLINE FINE-TUNING}\}$.

η : fed server learning rate.

```

1: Input: time  $t$ , client set  $\mathcal{S}_t$ , mode stage
2: if stage  $\in \{\text{OFFLINE PRETRAINING}, \text{REWARD TRAINING}\}$  then
3:   for all client  $k \in \mathcal{S}_t$  in parallel do
4:     Receive  $W_{t,k}$  from client  $k$ 
5:   end for
6:   Update:  $W^{t+1} \leftarrow$  synchronous aggregation
7:   Broadcast  $W^{t+1}$  to all clients
8: else if stage = ONLINE FINE-TUNING then
9:   Wait for incoming client update  $W_k^t$ 
10:  Update:  $W^{t+1} \leftarrow$  asynchronous aggregation
11:  Send  $W^{t+1}$  when the next client requests latest parameters
12: end if

```

B Proof of Theorem 1

B.1 Preliminaries

In this section, we introduce the theoretical basis of our privacy-preserving framework. We begin with the formal definition of (ϵ, δ) -differential privacy, which ensures that the presence or absence of a single data point has a limited effect on the algorithm's output.

DEFINITION B.1 (DIFFERENTIAL PRIVACY [8]). Let $\mathcal{A}$ be a randomized algorithm with domain $\mathcal{D}$ and range $\mathcal{R}$. We say that $\mathcal{A}$ satisfies (ϵ, δ) -differential privacy if, for any two neighboring datasets $D, D' \in \mathcal{D}$ differing in exactly one entry, and for any subset of outputs $S \subseteq \mathcal{R}$, it holds that:

$$\Pr[\mathcal{A}(D) \in S] \leq e^\epsilon \cdot \Pr[\mathcal{A}(D') \in S] + \delta. \quad (\text{B.11})$$

Here, $\epsilon > 0$ is the privacy budget that controls the worst-case privacy loss, and $\delta \in (0, 1)$ is a small parameter that bounds the probability of failure of the ϵ -privacy guarantee.

We next recall the Gaussian mechanism, a standard method for achieving differential privacy, which satisfies (ϵ, δ) -DP under appropriate parameters.

THEOREM B.1 (GAUSSIAN MECHANISM [9]). Let $f : \mathcal{D} \rightarrow \mathbb{R}^d$ be a function with ℓ_2 -sensitivity Δ_2 , that is, for any two neighboring datasets D and D' , it holds that:

$$\|f(D) - f(D')\|_2 \leq \Delta_2. \quad (\text{B.12})$$

The Gaussian mechanism adds Gaussian noise to the output of f as:

$$\mathcal{A}(D) = f(D) + \mathcal{N}(0, \sigma^2 I_d), \quad (\text{B.13})$$

where $\mathcal{N}(0, \sigma^2 I_d)$ is a d -dimensional Gaussian distribution with zero mean and covariance matrix $\sigma^2 I_d$.

When $\sigma \geq \frac{\sqrt{2 \log(1.25/\delta)} \cdot \Delta_2}{\epsilon}$, $\mathcal{A}$ satisfies (ϵ, δ) -differential privacy.

Lastly, we describe the Moment Accountant method, which tracks cumulative privacy loss by monitoring high-order moments.

Algorithm 3 Client Procedure with Lifecycle-Aware Training

Notation: $W_t^{\text{DT,E}}, W_t^{\text{DT,H}}$: parameters of global decision embedding and head model from fed server.
 $W_t^{\text{Critic,E}}, W_t^{\text{Critic,H}}$: parameters of global critic embedding and head model from fed server.
 $\mathcal{D}_k$: local expert trajectories collected for offline pretraining and reward training.

- 1: **Input:** time t , client k , stage
- 2: **if** stage = OFFLINE PRETRAINING **then**
- 3: Receive global parameters $W_{t-1}^{\text{DT,E}}, W_{t-1}^{\text{DT,H}}$ to update local decision models
- 4: Sample batch trajectories $(\mathbf{g}_k, \mathbf{s}_k, \mathbf{a}_k)$ from $\mathcal{D}_k$, predict action $\hat{\mathbf{a}}_k \leftarrow \text{DECISIONFORWARD}(\mathbf{g}_k, \mathbf{s}_k, \mathbf{a}_k)$
- 5: Compute decision loss $\mathcal{L}_k$ defined in Eq. (3), update $W_{t,k}^{\text{DT,E}}, W_{t,k}^{\text{DT,H}}$ through $\text{CLIENTBACKWARDANDUPDATE}(\mathcal{L}_k)$
- 6: Send $W_{t,k}^{\text{DT,E}}, W_{t,k}^{\text{DT,H}}$ to fed server for synchronous aggregation
- 7: **else if** stage = REWARD TRAINING **then**
- 8: Receive global parameters $W_{t-1}^{\text{Critic,E}}, W_{t-1}^{\text{Critic,H}}$ to update local critic models
- 9: Sample batch trajectories $(\mathbf{g}_k, \mathbf{s}_k, \mathbf{a}_k)$ from $\mathcal{D}_k$, calculate reward distribution $\mathcal{R}_k \leftarrow \text{DECISIONFORWARD}(\mathbf{s}_k, \mathbf{a}_k)$
- 10: Calculate the critic loss $\mathcal{L}_k$ defined in Eq. (4), update $W_t^{\text{Critic,E}}, W_t^{\text{Critic,H}}$ through $\text{CLIENTBACKWARDANDUPDATE}(\mathcal{L}_k)$
- 11: Send $W_{t,k}^{\text{Critic,E}}, W_{t,k}^{\text{Critic,H}}$ to fed server for synchronous aggregation
- 12: **else if** stage = ONLINE FINE-TUNING **then**
- 13: Receive global parameters $W_{t-1}^{\text{DT,E}}, W_{t-1}^{\text{DT,H}}, W_{t-1}^{\text{Critic,E}}, W_{t-1}^{\text{Critic,H}}$ to update all models. Initial trajectory $(\mathbf{g}_k, \mathbf{s}_k, \mathbf{a}_k = \text{None})$
- 14: **while** interaction is not done **do**
- 15: taking action $a \leftarrow \text{DECISIONFORWARD}(\mathbf{g}_k, \mathbf{s}_k, \mathbf{a}_k)$
- 16: Obtain environment reward r and state s from VIRTUALTAOBAO, update trajectory $(\mathbf{g}_k, \mathbf{s}_k, \mathbf{a}_k)$ by concatenating $(a, g_{-1} - r, s)$
- 17: **end while**
- 18: Add new interaction trajectory $(\mathbf{g}_k, \mathbf{s}_k, \mathbf{a}_k)$ to ReplayBuffer_k
- 19: **if** $|\text{ReplayBuffer}_k| \geq N_{\text{finetune}}$ **then**
- 20: Sample batch trajectories $(\mathbf{g}_k, \mathbf{s}_k, \mathbf{a}_k)$ from ReplayBuffer_k , predict action $\hat{\mathbf{a}}_k \leftarrow \text{DECISIONFORWARD}(\mathbf{g}_k, \mathbf{s}_k, \mathbf{a}_k)$
- 21: Calculate reward distribution $\mathcal{R}_k \leftarrow \text{DECISIONFORWARD}(\mathbf{s}_k, \hat{\mathbf{a}}_k)$
- 22: Calculate the loss $\mathcal{L}_k$ in Eq. (7), update $W_t^{\text{DT,E}}, W_t^{\text{DT,H}}, W_t^{\text{Critic,E}}, W_t^{\text{Critic,H}}$ through $\text{CLIENTBACKWARDANDUPDATE}(\mathcal{L}_k)$
- 23: Send $W_t^{\text{DT,E}}, W_t^{\text{DT,H}}, W_t^{\text{Critic,E}}, W_t^{\text{Critic,H}}$ to fed server for asynchronous aggregation
- 24: **end if**
- 25: **end if**
- 26: **function** $\text{DECISIONFORWARD}(\mathbf{g}, \mathbf{s}, \mathbf{a})$
- 27: Compute decision embeddings with DP noise $\tilde{\mathbf{E}}_k$ by decision embedding model
- 28: Send $\tilde{\mathbf{E}}_k$ to compute server and then receive hidden decision state $\mathbf{H}_k$ from compute server, predict action $\hat{\mathbf{a}}_k$ by decision head model
- 29: **return** predicted action $\hat{\mathbf{a}}_k$
- 30: **end function**
- 31: **function** $\text{CRITICFORWARD}(\mathbf{s}, \mathbf{a})$
- 32: Compute critic embeddings with DP noise $\tilde{\mathbf{C}}_k$ by critic embedding model
- 33: Send $\tilde{\mathbf{C}}_k$ to compute server and receive hidden critic state $\mathbf{G}_k$ from compute server
- 34: Estimate reward distribution $\mathcal{R}_k$ by critic head model
- 35: **return** reward distribution $\mathcal{R}_k$
- 36: **end function**
- 37: **function** $\text{CLIENTBACKWARDANDUPDATE}(\mathcal{L}_k)$
- 38: Backpropagate $\mathcal{L}_k$ through head and embedding models with gradient clipping and noise: $\nabla_k \leftarrow \nabla_k + \mathcal{N}(0, \sigma_{\text{grad}}^2 \mathbf{I})$
- 39: Update: $W_k \leftarrow W_k - \eta \cdot \nabla_k$
- 40: **end function**

THEOREM B.2 (MOMENTS ACCOUNTANT [1]). *There exist constants c_1 and c_2 so that given the sampling probability $q = \frac{L}{N}$ and the number of steps T , for any $\epsilon < c_1 q^2 T$, DP-SGD is (ϵ, δ) -differentially private for any $\delta > 0$ if we choose: $\sigma \geq c_2 \frac{q\sqrt{T \log(1/\delta)}}{\epsilon}$.*

B.2 Privacy Guarantee

LEMMA B.1 (PRIVACY GUARANTEE VIA GAUSSIAN NOISE CALIBRATION). *There exist constants c_1 and c_2 such that, given a sample rate b_L , and letting B denote the local batch size, L the number of sampled clients per round, E the number of differential privacy events per round, R the total number of rounds, N_k the number of local samples on each client, and K the total number of clients, the mechanism FeedGuard satisfies (ϵ_L, δ) -differential privacy for*

any $\delta > 0$, as long as $\epsilon_L < c_1 b_L^2 ER$, provided that we choose:

$$\sigma_L \geq c_2 \frac{\left(\frac{B}{N_k} \cdot \frac{1}{K}\right) \sqrt{ER \log(1/\delta)}}{\epsilon_L}.$$

PROOF. We begin by analyzing the privacy budget associated with the clients selected during training under this end-to-end framework. To maintain notational consistency, we let N denote the total number of training samples. According to analysis in [28], in the client-server architecture, there exist constants c_1 and c_2 such that, for any $\epsilon_L < c_1 b_L^2 T_L$, the following holds: $\sigma_L \geq c_2 \frac{b_L \sqrt{T_L \log(1/\delta)}}{\epsilon_L}$. Under our framework settings, b_L , the sampling rate of each selected client, satisfies $b_L = \frac{B}{N}$. ϵ_L denotes the privacy budget assigned to each client. For analytical convenience, we approximate $\epsilon_L \approx \frac{\epsilon}{L}$, where ϵ is the total privacy budget. T_L represents the number of local training steps, given by $T_L = ER$. Note that in our framework, we set each round to correspond to a single epoch.

Substituting these into the above equation, we obtain: $\sigma_L \geq c_2 \frac{\frac{B}{N} \sqrt{ER \log(1/\delta)}}{\epsilon_L} = c_2 \frac{\left(\frac{B}{N_k} \cdot \frac{1}{K}\right) \sqrt{ER \log(1/\delta)}}{\epsilon_L}$. Note that $\frac{B}{N_k}$ corresponds to the sample-level sampling rate, while $\frac{1}{K}$ indicates the client-level sampling rate in federated learning. $\square$

B.3 Lifecycle Privacy Budget Allocation

This section proves the allocation strategy in Theorem 1.

PROOF. Our framework consists of three stages: Offline Pretraining (denoted as off), Asymmetric Reward Training (denoted as rew), and Online Fine-tuning (denoted as on). Based on Theorem B.2, we first derive the allowable standard deviation of the Gaussian noise in each stage. During the Offline Pretraining stage, we obtain: $\sigma_{\text{off}} \geq c_{\text{off}} \frac{q \sqrt{T_{\text{off}} \log(1/\delta)}}{\epsilon_{\text{off}}}$. Here, ϵ_{off} denotes the privacy budget allocated to this stage, c_{off} is a constant, q represents the uniform sampling rate, T_{off} indicates the number of training steps, and δ corresponds to the failure probability, i.e., the maximum probability that the algorithm violates strict ϵ -differential privacy.

Similarly, in the Asymmetric Reward Training stage, $\sigma_{\text{rew}} \geq c_{\text{rew}} \frac{q \sqrt{T_{\text{rew}} \log(1/\delta)}}{\epsilon_{\text{rew}}}$, and in the Online Fine-tuning stage, $\sigma_{\text{on}} \geq c_{\text{on}} \frac{q \sqrt{T_{\text{on}} \log(1/\delta)}}{\epsilon_{\text{on}}}$. For simplicity, we define a unified constant $c = \max\{c_{\text{off}}, c_{\text{rew}}, c_{\text{on}}\}$ across the three stages. Assuming each stage has R rounds, the number of training steps in each stage is given by: $T_{\text{off}} = 3R$, $T_{\text{rew}} = 3R$, $T_{\text{on}} = 6R$. Consequently, the training stages satisfy the relationship: $\sqrt{T_{\text{off}}} : \sqrt{T_{\text{rew}}} : \sqrt{T_{\text{on}}} = 1 : 1 : \sqrt{2}$.

Furthermore, from a global perspective, the entire framework is designed to satisfy end-to-end (ϵ, δ) -differential privacy. To this end, the total privacy budget ϵ is allocated to the three stages, namely ϵ_{off} , ϵ_{rew} , and ϵ_{on} , such that the Gaussian noise standard deviation is consistent across stages, denoted by σ . Therefore, by combining the relationships derived above, we obtain: $\frac{\sqrt{T_{\text{off}}}}{\epsilon_{\text{off}}} = \frac{\sqrt{T_{\text{rew}}}}{\epsilon_{\text{rew}}} = \frac{\sqrt{T_{\text{on}}}}{\epsilon_{\text{on}}}$.

Here, we introduce a set of allocation weights to characterize the relationship between the total privacy budget and the privacy budgets assigned to the three stages:

$$\epsilon_{\text{off}} = \lambda_{\text{off}} \epsilon, \quad \epsilon_{\text{rew}} = \lambda_{\text{rew}} \epsilon, \quad \epsilon_{\text{on}} = \lambda_{\text{on}} \epsilon, \quad \lambda_{\text{off}} + \lambda_{\text{rew}} + \lambda_{\text{on}} = 1. \quad (\text{B.14})$$

Based on the previous equations, we obtain the allocation weights

$$\lambda_{\text{off}} = \lambda_{\text{rew}} = \frac{1}{2+\sqrt{2}} \text{ and } \lambda_{\text{on}} = \frac{\sqrt{2}}{2+\sqrt{2}}. \quad \square$$

C Datasets and Implement Details

Dataset. For offline experiments, we use four datasets: MovieLens 1M¹, Amazon CD², MovieLens 20M³, and Netflix⁴, covering movie and e-commerce recommendation. Dataset statistics are summarized in Table 1. For online experiments, we utilize VirtualTB [26], a simulator constructed from large-scale Taobao user logs, to simulate dynamic interactions with sequential decision-making and policy shifts. It incorporates action norm constraints to enhance generalization. For offline pretraining, we train an expert agent using the Deep Deterministic Policy Gradient (DDPG) algorithm in VirtualTB and collect user interaction trajectories. Each client contributes 1,000 expert trajectories, forming the offline buffer.

Model Architecture. On the client side, embedding and head layers are implemented as linear layers, with an embedding dimension of 128. The decision head outputs actions matching the VirtualTB action space, while the critic head predicts reward parameters via a one-dimensional Gaussian distribution. On the compute server side, the decision model follows the Decision Transformer [5] architecture with 6 layers, 8 attention heads, and a hidden dimension of 128. The critic model is a lighter causal Transformer with 3 layers, 2 attention heads, and the same hidden dimension.

Federated Learning and Privacy Settings. We simulate a federated setup with 1000 clients, where in each communication round, a random subset of 10% clients (sampling rate = 0.1) is selected to participate in training. To enforce differential privacy, we adopt the Opacus framework⁵ for per-sample gradient clipping and noise injection. Each selected client performs local training with a privacy sampling rate of 0.1, meaning only 10% of its local data is used per update for privacy-preserving training. Gradients are clipped to a maximum norm of $\max_grad_norm = 1.0$, and Gaussian noise is added based on a target privacy budget (ϵ, δ) , where we fix $\delta = 10^{-5}$. The noise multiplier is computed according to our theoretical framework and varies with $\epsilon \in \{11, 20, 30, 40, 50\}$. Smaller values of ϵ correspond to stricter privacy guarantees and therefore require larger noise multipliers. For example, $\epsilon = 11$ corresponds to a noise multiplier of approximately 20, which is near the practical upper limit of what the model can tolerate.

Training and Evaluation. The learning rate for both client and server models is tuned in $[1e-4, 1e-2]$, while two loss hyperparameters, λ and α , are selected from 1, 0.1, 0.01, 0.001. After pretraining, models are evaluated and fine-tuned in the VirtualTB simulator for 10^5 online episodes. Each episode represents an interaction session with a randomly sampled user. The click-through rate (CTR) is recorded every 50 episodes to track online performance. All experiments are conducted on four NVIDIA RTX 4090 GPUs (24GB VRAM each). Offline pretraining, reward training, and online fine-tuning take approximately 1, 0.75, and 13 GPU hours, respectively.

¹<https://grouplens.org/datasets/movielens/1m/>

²<https://nijianmo.github.io/amazon/index.html>

³<https://grouplens.org/datasets/movielens/20m/>

⁴<https://www.kaggle.com/datasets/netflix-inc/netflix-prize-data>

⁵<https://github.com/pytorch/opacus>